ABSTRACT

This project automates collection, processing, and reporting of security data from platforms like Sumo Logic, SecurityScorecard, Cisco Umbrella, and Trend Vision One. Traditional SOC environments rely on manual log collection, which is time-consuming and error-prone. This project eliminates manual intervention using Python scripting and automation tools. Selenium enables automated login, navigation, data extraction, and screenshot capture. SecurityScorecard data includes risk overview and score factors. Cisco Umbrella is integrated through automated screenshot capture based on predefined instructions. Sumo Logic provides both automated screenshot capture of user and role configurations, as well as log collection using predefined queries. Trend Vision One contributes advanced threat intelligence and detection insights, which are incorporated into the consolidated reporting process. Screenshots and CSV files are stored and integrated into PowerPoint reports.CSV files are parsed to extract required fields and arranged in a standard format. The workflow runs weekly via Task Scheduler, ensuring timely execution. Weekly data is consolidated into accurate monthly security reports. Automation reduces manual effort, improves efficiency, and enhances SOC productivity

LIST OF FIGURES

CONTENTS

CHAPTER 1

INTRODUCTION

In modern cybersecurity environments, Security Operations Center (SOC) teams play a critical role in monitoring, detecting, and responding to security threats. These teams rely on multiple security platforms such as **Sumo Logic, Security Scorecard, Cisco Umbrella, and Trend Vision One** to collect logs, analyze risks, and generate security reports. However, in traditional SOC workflows, the process of collecting data and preparing reports is largely manual and repetitive. Analysts must log into each platform, extract relevant information, capture screenshots, and organize the data into structured reports. This manual approach consumes significant time and increases the chances of human errors and delays in report generation.

To address these challenges, this project proposes an **Automated Data Collection and Report Generation System** using Python-based automation techniques. The system automatically logs into multiple cybersecurity platforms, extracts required data, captures screenshots, and downloads logs in structured formats such as CSV files. A task scheduler is used to run the automation process at predefined intervals, such as weekly or monthly, ensuring timely and consistent data collection.

After the data is collected, it is processed and organized into a standardized format, and the system generates a well-structured report in the form of a PowerPoint presentation. This automation reduces manual effort, improves efficiency, and ensures consistency and accuracy in SOC reporting. Ultimately, the system enables SOC analysts to focus more on critical security tasks such as threat detection and incident response, thereby improving overall operational effectiveness.

1.1 MOTIVATION FOR THE WORK

In Security Operations Center (SOC) environments, generating reports from multiple security tools is repetitive, time-consuming, and prone to errors, leading to inefficiencies in daily operations.

This project is motivated by the need to automate data collection and report generation to reduce manual effort, improve accuracy, and enable SOC teams to focus on critical security tasks such as threat analysis and incident response.

1.2 OBJECTIVE OF THE WORK

The main objective of this project is to design and develop an automated system that simplifies SOC reporting processes. The key objectives include,Automating data collection from multiple cybersecurity platforms.Reducing manual effort and minimizing human errors.Standardizing report generation for consistency.Improving efficiency and saving time in SOC operations.Integrating logs, screenshots, and risk data into a unified report.Enabling scheduled execution of the automation process.This system ensures that SOC teams can generate accurate and timely reports with minimal effort, thereby enhancing overall operational effectiveness.

CHAPTER 2

LITERATURE SURVEY

Authors: Paruchuri Ramya; Vemuri Sindhura; P. Vidya Sagar
Title: *Testing using Selenium Webdriver*

Software testing is a crucial phase in the Software Development Life Cycle (SDLC) that ensures software meets user requirements and produces expected results. Manual testing can be time-consuming, which has led to the adoption of test automation to improve efficiency and accuracy. A study demonstrated the use of **Selenium WebDriver** for automating web application testing and highlighted its integration with tools like **Maven** and **TestNG** to simplify the testing process and enhance software quality. This study is relevant to the proposed project, as Selenium is used to automate tasks such as logging into systems, extracting data, capturing screenshots, and generating reports efficiently.
Published in: 2017 Second International Conference on Electrical, Computer and Communication Technologies (ICECCT)

Authors: Archana PS; Christan Jose; Albin Biju; Aromal Dileep; Aswanth G Pillai
Title: *An Integrated SOC Tool*

This paper presents a comprehensive survey of existing SOC tools and identifies key challenges such as lack of integration, high false positives, and excessive manual intervention. The study emphasizes the importance of automation, machine learning, and real-time data visualization in improving SOC efficiency. It discusses various methodologies including Zero Trust architecture, SDN/NFV-based security models, and machine learning-based anomaly detection for enhancing threat detection capabilities. The paper also highlights the limitations of current tools like SIEM and SOAR, such as scalability issues and alert overload. It proposes the development of an integrated SOC tool with features like automated threat detection, customizable dashboards, and seamless integration with existing infrastructure to improve overall cybersecurity operations.
Published in: December-2024 International Journal of Innovative Science and Research Technology

Author: Ajayi Toluwalope

Title: *Automating Security Operations Centers (SOCs) with AI: Benefits and Challenges*

This paper explores the role of artificial intelligence in enhancing SOC operations by automating threat detection, incident response, and data analysis. The study highlights that AI-driven systems can significantly reduce detection and response time by analyzing large volumes of security data and identifying anomalies in real time. It also discusses the advantages of AI such as improved efficiency, scalability, and reduced workload for security analysts. However, the paper addresses several challenges including false positives, adversarial attacks, dependency on high-quality training data, and ethical concerns in automated decision-making. The research concludes that while AI can greatly enhance SOC performance, it must be combined with human oversight and continuous model updates to ensure reliability and effectiveness.

Published in: (8th February, 2025)

Author: Elis Pelivani ; Betim Cico

Title: *A comparative study of automation testing tools for web applications*

In recent years, websites and web applications have become essential for businesses and organizations, making software development and testing a critical part of ensuring reliable and error-free systems. Software testing plays a significant role in the Software Development Life Cycle (SDLC) as it helps maintain the quality, consistency, and performance of applications. Traditionally, testing was performed manually; however, manual testing is time-consuming, costly, and prone to human errors. As a result, many organizations have shifted towards automated testing methods to improve efficiency and reduce operational costs.

A study was conducted to evaluate and compare different automation testing tools used for web application testing, particularly **Katalon Studio** and **Selenium**. The paper analyzed various features of these tools, including usability, flexibility, performance, and cost-effectiveness. The findings highlighted that automation tools significantly reduce testing time, improve accuracy, and enhance the overall quality of software applications.

Published in: 2021 10th Mediterranean Conference on Embedded Computing (MECO), (12 June, 2021)

Author: S. Ganesh ; A. Padmini Celestina ; R. Jayashree ; K.V. HariPriya

Title: *Web Automation in Healthcare*

Modern business applications are widely built on web-based systems that handle large amounts of data and require extensive processing. Many tasks such as form filling, data extraction, screen scraping, and periodic report generation are repetitive and time-consuming when performed manually.

Web automation helps reduce manual effort, save time, and improve operational efficiency. A study on **Robotic Process Automation (RPA)** in the healthcare industry showed that software bots can mimic human actions such as logging into applications, entering data, performing calculations, and completing routine tasks automatically. The implementation of automation significantly reduced workload and improved accuracy and productivity.

This study is relevant to the proposed project, as it demonstrates how automation can be used to handle repetitive web-based tasks. Similarly, this project uses automation to log into multiple systems, collect data, capture screenshots, and generate periodic reports efficiently.

Published in: 2019 IEEE International Conference on Innovations in Communication, Computing and Instrumentation (ICCI) (23 March,2019)

CHAPTER 3

SYSTEM DESIGN

3.1 INTRODUCTION

In modern Security Operations Centers (SOCs), security analysts rely on multiple platforms such as Sumo Logic, Security Scorecard, Cisco Umbrella, and Trend Vision One to monitor and analyze security data. However, collecting data from these platforms manually is time-consuming, repetitive, and prone to human error.

To overcome these challenges, this project introduces an automated system that streamlines data collection, processing, and report generation. The system leverages Python automation and Selenium to extract relevant security data, capture screenshots, and download logs from various platforms. The collected data is then organized and compiled into structured reports for easy analysis.

The system is designed to run on a scheduled basis, ensuring consistent and timely report generation without manual intervention. By automating repetitive SOC tasks, the system enhances efficiency, reduces workload, and allows analysts to focus on critical threat detection and response activities.

3.2 SOFTWARE REQUIREMENTS

The system requires the following software tools and technologies:

- **Python:** Used for automation, scripting, and backend processing.
- **Selenium:** Enables automated web interaction such as logging into platforms and capturing screenshots.
- **Microsoft Excel:** Used for organizing and analyzing collected data.
- **Microsoft PowerPoint:** Used for generating automated reports and dashboards.
- **Web Browsers (Chrome/Edge):** Required for accessing SOC platforms during automation.
- **Task Scheduler:** Used to automate execution of scripts at predefined intervals.

3.3 SELENIUM

Selenium is an open-source web automation tool used to control and interact with web applications through a browser programmatically. It allows developers to automate tasks such as logging into websites, navigating web pages, clicking buttons, filling forms, extracting data, and capturing screenshots. In this project, Selenium is implemented using **Python** to automate repetitive activities performed by Security Operations Center (SOC) analysts.

In the **Automated Data Collection and Report Generation for SOC** project, Selenium plays a key role in automating the process of accessing multiple cybersecurity platforms such as **Sumo Logic, Security Scorecard, Cisco Umbrella, and Trend Vision One**. Instead of manually logging into each platform, Selenium scripts automatically open the web browser, enter login credentials, and navigate to specific dashboards or report sections. The tool then extracts required information, captures screenshots of important security metrics, and downloads logs or reports in formats such as CSV or PDF.

Additionally, Selenium is integrated with scheduling mechanisms so that the automation process can run at predefined intervals, such as weekly or monthly. This ensures that data collection and report generation occur consistently without manual intervention. The captured screenshots and downloaded data are later used to generate structured PowerPoint reports as part of the automated workflow.

Overall, Selenium significantly improves efficiency by reducing manual effort, minimizing human errors, and speeding up the data collection process. Its use in this project ensures reliable, repeatable, and timely generation of SOC reports, allowing security analysts to focus more on threat detection and incident response rather than routine documentation tasks

3.4 TASK SCHEDULER

, Windows Task Scheduler plays a critical role by ensuring the automation workflow runs reliably and consistently each week without manual intervention. It is configured to trigger the Python script at predefined times, launching the automation sequence that logs into platforms like Sumo Logic, SecurityScorecard, Cisco Umbrella, and Trend Vision One, captures screenshots, collects logs where applicable, and stores CSV outputs. By handling execution automatically, Task Scheduler guarantees timely data collection and reporting, supports error handling through retry options, and maintains logs for troubleshooting. This

integration ensures that weekly tasks are executed seamlessly, enabling accurate consolidation into monthly security reports and significantly improving SOC efficiency and productivity

.

3.5 EXISTING SYSTEM

In traditional SOC environments, data collection and report generation are performed manually by security analysts. This process involves logging into multiple platforms individually, navigating through different dashboards, extracting relevant information, capturing screenshots, and compiling reports manually. Such a workflow is not only time-consuming but also prone to errors due to repetitive manual operations.

The existing system lacks automation and integration across different platforms, making it difficult to maintain consistency and accuracy in reporting. Analysts often spend a significant amount of time on routine tasks instead of focusing on critical activities such as threat analysis and incident response. Moreover, the absence of standardized reporting formats leads to inconsistencies in documentation, further reducing efficiency. Although some tools provide limited automation capabilities, they do not offer a unified solution that integrates multiple platforms and automates the entire workflow effectively.

3.6 PROPOSED SYSTEM

The proposed system introduces an automated approach to data collection and report generation in SOC environments, addressing the limitations of the existing system. The system is designed to automatically interact with multiple security platforms using Selenium-based automation. It logs into each platform, retrieves relevant data, captures screenshots of dashboards, and downloads logs where necessary. The collected data is then processed and organized into structured formats for reporting.

One of the key features of the proposed system is its ability to generate reports automatically in a standardized format using PowerPoint. This ensures consistency and improves the clarity of the reports. Additionally, the system is integrated with a scheduling mechanism that allows it to run at predefined intervals, such as weekly or monthly, without requiring manual intervention. This automation not only reduces the workload of SOC analysts but also improves accuracy and efficiency. By providing a unified and automated solution, the

proposed system enhances the overall effectiveness of security operations and enables analysts to focus on more critical tasks.

3.7 TERMINOLOGIES USED

- **Security Operations Center (SOC)** – A centralized unit responsible for monitoring, detecting, and responding to cybersecurity threats.
- **Automation** – The process of performing tasks using software scripts without manual intervention.
- **Selenium** – A tool used for automating web browser interactions such as logging into platforms and extracting data.

- **Log Data** – Records generated by systems and applications that provide insights into system activities and potential security events.
- **Dashboard** – A visual interface that displays important metrics and information related to system security.
- **Task Scheduling** – The process of executing scripts automatically at predefined times to ensure continuous system operation.

3.8 DIAGRAM

3.8.1 BLOCK DIAGRAM

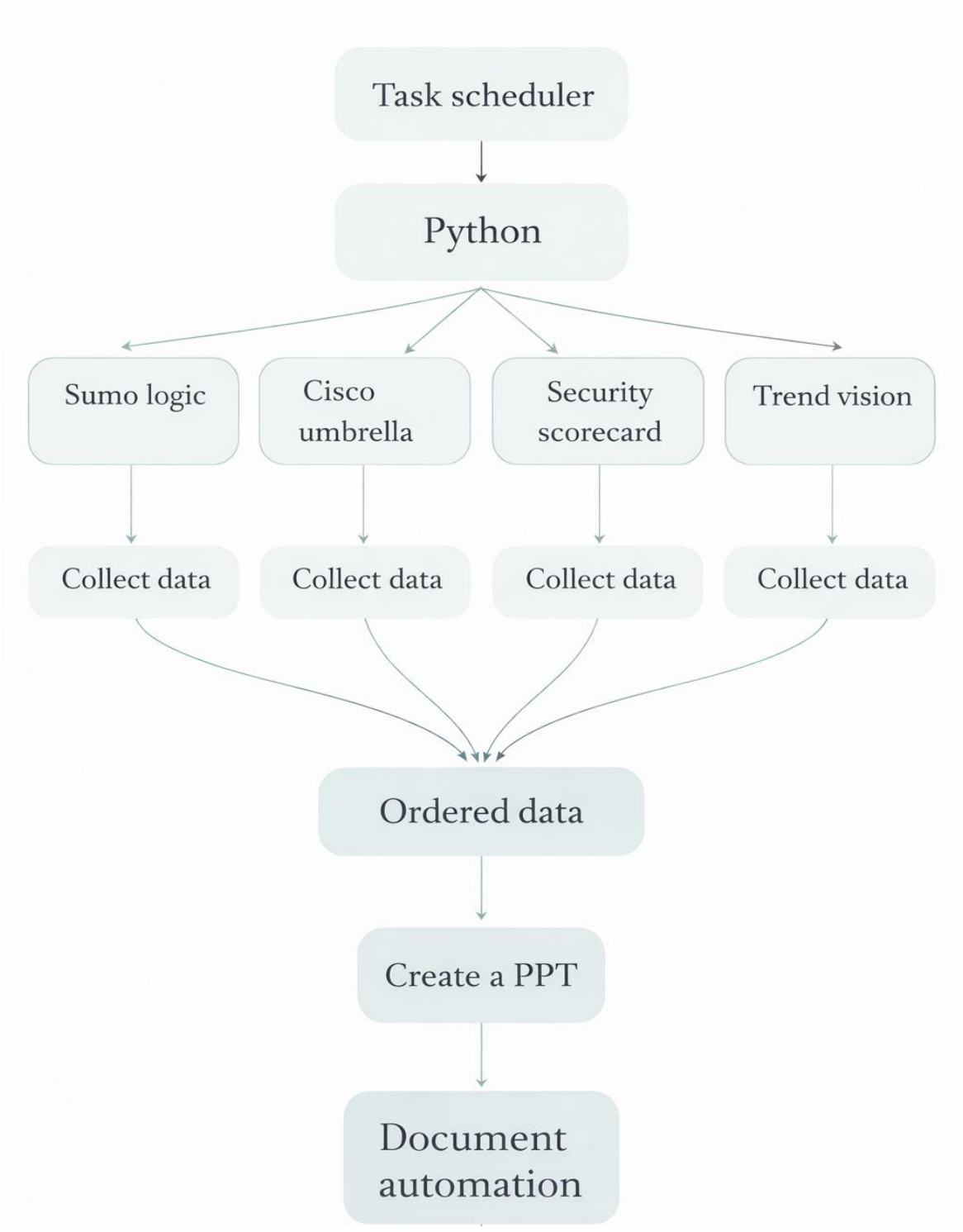


Fig 3.8.1 Block diagram

3.8.2 OVERALL SYSTEM WORKFLOW

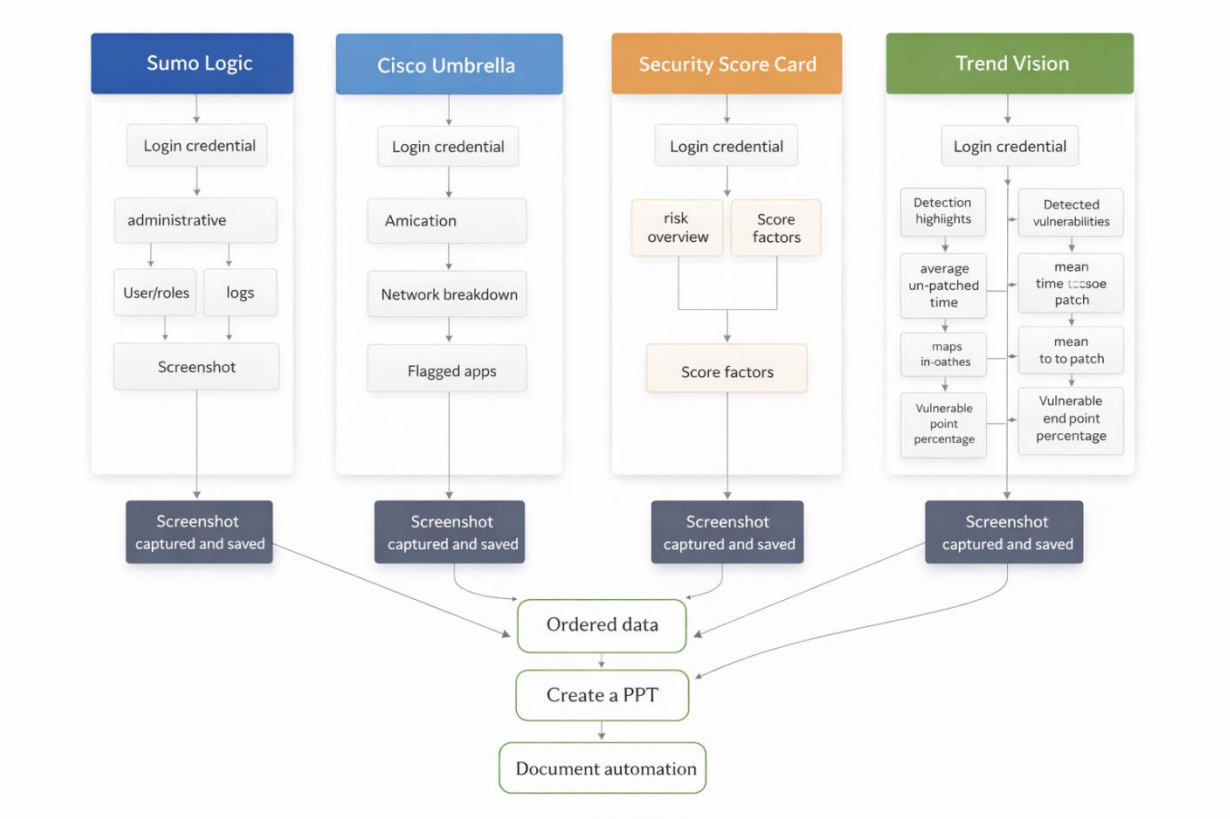


Fig 3.8.2 Overall System Workflow

3.9 TESTING

Testing plays a crucial role in ensuring the reliability and efficiency of the system. It helps identify errors and ensures that all components function as expected under different conditions.

3.9.1 MODULE WISE TESTING

Module-wise testing involves testing each component of the system individually to verify its functionality. This includes testing the login automation module to ensure successful authentication, the data extraction module to verify accurate data collection, the screenshot capture module to confirm proper image capture, and the report generation module to ensure correct formatting and content in the generated reports.

3.9.2 APPLICATION TESTING

Application testing involves evaluating the entire system as a whole to ensure seamless integration of all modules. This includes verifying the complete workflow from data collection to report generation, checking the accuracy and consistency of the output, and ensuring that the scheduling mechanism functions correctly. Through comprehensive testing, the system is validated for real-world usage in SOC environments.

CHAPTER 4

MODULE DESCRIPTION

4.1 AUTOMATION SETUP & AUTHENTICATION

The automation system begins with the authentication process, which enables secure access to multiple cybersecurity platforms. The system uses automated login mechanisms implemented through Python and Selenium, allowing it to enter credentials such as username and password without manual intervention. This ensures that the automation process can seamlessly access platforms like Sumo Logic, Security Scorecard, Cisco Umbrella, and Trend Vision One.

To maintain efficiency, session handling is implemented so that repeated logins are avoided during execution. Once authentication is successful, the system navigates to the required dashboards of each platform. This module plays a crucial role in ensuring uninterrupted access to data sources and forms the foundation for the entire automation workflow.

4.2 DATA COLLECTION SYSTEM

The data collection module is responsible for extracting relevant security information from multiple platforms. It automates the process of accessing different sections within each tool and retrieving the required data for reporting purposes.

4.2.1 PLATFORM DATA EXTRACTION

In this module, the system collects data from various SOC tools. For Sumo Logic, the automation script logs into the platform, navigates to the logs section, executes predefined queries, and exports the results in CSV format. In Security Scorecard, the system captures risk scores and security ratings, providing an overview of the organization’s security posture.

Similarly, in Cisco Umbrella, the system extracts information related to application usage and network activity, while in Trend Vision One, it gathers data related to threat detection, vulnerabilities, and endpoint security metrics. By integrating multiple platforms, this module ensures comprehensive data collection from diverse sources.

4.2.2 SCREENSHOT CAPTURE SYSTEM

In addition to extracting raw data, the system captures screenshots of important dashboards and visual panels. This includes risk overview pages, threat detection dashboards, and log analysis screens. These screenshots provide visual evidence and enhance the clarity of the final report.

The screenshot capture process is fully automated and ensures that all relevant information is documented without manual effort. This feature is particularly useful for audit purposes and improves the overall quality of the generated reports.

4.3 DATA PROCESSING & MANAGEMENT

4.3.1 DATA PROCESSING

Once the data is collected, it is processed to ensure consistency and usability. The system converts raw data into a structured format, making it easier to analyze and integrate into reports. Unnecessary or duplicate data is removed, and relevant fields are organized systematically.

This step ensures that the final output is clean, accurate, and ready for presentation. Proper data processing is essential for maintaining the reliability of the automation system.

4.3.2 DATA STORAGE

After processing, the data is stored in an organized manner for future use. The system maintains separate storage for CSV files, screenshots, and processed data. Proper file management ensures easy retrieval and helps maintain a history of previous reports.

This module plays an important role in maintaining data integrity and supporting long-term analysis of security trends.

4.4 REPORT GENERATION SYSTEM

4.4.1 AUTOMATED REPORT CREATION

The report generation module is responsible for creating structured reports based on the collected and processed data. The system automatically generates reports in PowerPoint format, combining data, screenshots, and summary insights into a single document.

This eliminates the need for manual report preparation and ensures that reports are generated quickly and efficiently. The automation ensures consistency in formatting and content across all reports.

4.4.2 REPORT FORMATTING

The generated reports follow a standardized layout, ensuring uniformity and professional presentation. Each report includes sections such as data summaries, visual dashboards, and key observations. The system ensures proper alignment, spacing, and formatting to enhance readability.

By maintaining a consistent structure, this module improves the quality and usability of SOC reports.

4.5 TASK SCHEDULING SYSTEM

4.5.1 SCHEDULER CONFIGURATION

The task scheduling module enables the automation process to run at predefined intervals without manual intervention. The system uses scheduling tools such as Task Scheduler to trigger the execution of the automation script.

This ensures that reports are generated regularly, such as on a weekly or monthly basis, without requiring user input.

4.5.2 SCHEDULING AND EXECUTION FLOW

Once configured, the scheduler automatically initiates the automation process at the specified time. The script then performs all operations, including login, data collection, processing, and report generation.

This module significantly reduces manual workload and ensures timely report generation, making it highly efficient for SOC environments.

4.6 OUTPUT & DELIVERY SYSTEM

4.6.1 OUTPUT GENERATION

The final module handles the generation of outputs, which include PowerPoint reports, CSV files, and captured screenshots. These outputs provide a complete view of the collected security data and analysis.

4.6.2 FILE MANAGEMENT AND STORAGE

The system organizes all generated outputs into structured folders, ensuring easy access and retrieval. It also maintains a history of reports, allowing users to review past data and track changes over time.

CHAPTER 5

CODING IMPLEMENTATION

```
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.keys import Keys
from selenium.common.exceptions import TimeoutException
from PIL import Image
from selenium.webdriver.common.action_chains import ActionChains

import os
import time
import datetime


# ---------------- CHROME SETUP ----------------
chrome_options = Options()
chrome_options.add_argument("--start-maximized")
chrome_options.add_experimental_option("detach", True)

driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=chrome_options
)
wait = WebDriverWait(driver, 20)

# ---------------- OPEN SITE ----------------
driver.get("https://www.sumologic.com")

# ---------------- START FREE TRIAL ----------------
start_trial_btn = wait.until(
    EC.element_to_be_clickable((
        By.XPATH,
        "//a[@href='https://www.sumologic.com/sign-up' and text()='Start free trial']"
    ))
)
driver.execute_script("arguments[0].scrollIntoView(true);", start_trial_btn)
time.sleep(1)
```

```
driver.execute_script("arguments[0].click();", start_trial_btn)
print("Correct Start Free Trial Button Clicked")

# ---------------- SIGN IN ----------------
sign_in = wait.until(
    EC.element_to_be_clickable((
        By.XPATH,
        "//a[contains(text(),'Sign In') or contains(text(),'Sign in')]"
    ))
)
driver.execute_script("arguments[0].click();", sign_in)
print("Sign in clicked")

email = driver.find_element(By.NAME, "email")
email.send_keys("seshaselvam07@gmail.com")

password = driver.find_element(By.NAME, "password")
password.send_keys("#Iamsesha1")

sign_in_btn = wait.until(
    EC.element_to_be_clickable((By.XPATH, '//*[@id="signin-button"]'))
)
driver.execute_script("arguments[0].click();", sign_in_btn)
print("Signed in successfully")
time.sleep(3)

# ---------------- ADMINISTRATION ----------------
admin_button = wait.until(
    EC.presence_of_element_located((By.XPATH,'//*[@id="kanso-nav-item-
administration"]/div/div[2]'))
)

driver.execute_script("arguments[0].scrollIntoView(true);", admin_button)
time.sleep(1)
driver.execute_script("arguments[0].click();", admin_button)
print("Administration menu item clicked successfully")


# ---------------- SAFE CLICK FUNCTION ----------------
def safe_click(xpath):
    el = wait.until(EC.presence_of_element_located((By.XPATH, xpath)))
    driver.execute_script("arguments[0].scrollIntoView(true);", el)
    time.sleep(0.5)
    driver.execute_script("arguments[0].click();", el)
    print(f"Clicked element: {xpath}")
```

```
# ---------------- WEEK LABEL FUNCTION ----------------
def get_week_label(tool_name, page_no=""):
    today = datetime.date.today()
    week_no = (today.day - 1) // 7 + 1


suffix = ['st', 'nd', 'rd', 'th'][min(week_no - 1, 3)]
    return

f"{week_no}{suffix}week_{today.strftime('%b').lower()}_{tool_name}{page_no}.png"


# ---------------- CROP FUNCTION ----------------
def crop_and_save(input_file, crop_box, tool_name, page_no=""):
    img = Image.open(input_file)

    cropped = img.crop(crop_box)

    filename = get_week_label(tool_name, page_no)
    cropped.save(filename)

    print(f"Cropped & Saved → {filename}")

    img.close()
    os.remove(input_file)


# Roles and Users menu
safe_click('//*[@id="sumoapp"]/div/div/div/div[1]/div/nav/div[2]/div/div/div[1]/ul/div[2]/div[2]
/div/h3/button')
safe_click('//*[@id="kanso-nav-item-users"]/div/div')

# # ---------------- WAIT FOR PAGE LOAD ----------------
time.sleep(3)
driver.execute_script("""
    /* Hide the left blue sidebar */
    const sidebar = document.querySelector('nav');
    if (sidebar) {
        sidebar.style.display = 'none';
    }

    /* Expand main content */
    const mainContent = document.querySelector('main');
    if (mainContent) {
        mainContent.style.marginLeft = '0px';
        mainContent.style.width = '100%';
    }
```

```
    /* Fix wrapper spacing */
    const appRoot = document.querySelector('#sumoapp');
    if (appRoot) {
        appRoot.style.marginLeft = '0px';
        appRoot.style.paddingLeft = '0px';
    }


    /* Remove any leftover grid offset */
    document.body.style.overflowX = 'hidden';
""")

# ---------------- LIST TO STORE SCREENSHOT PATHS ----------------
screenshot_paths = []


# ---------------- SCREENSHOT FUNCTION ----------------
def take_users_table_screenshot(page_no):
    users_table = wait.until(
        EC.visibility_of_element_located((
            By.XPATH,
            "//table | //div[contains(@class,'ReactTable')] | //div[contains(@class,'users')]"
        ))
    )

    driver.execute_script("arguments[0].scrollIntoView(true);", users_table)
    time.sleep(1)

    driver.execute_script("arguments[0].style.zoom='60%'", users_table)
    time.sleep(1)

    # TEMP full screenshot
    raw_file = f"sumo_raw_page{page_no}.png"
    driver.save_screenshot(raw_file)

    print(f"Full screenshot saved → {raw_file}")

    # CROP (adjust values once based on your screen)
    crop_and_save(
        input_file=raw_file,
        crop_box=(0, 0, 900, 880),  # remove left menu + header
        tool_name="sumologic",
        page_no=page_no
    )

time.sleep(10)
# ---------------- TAKE SCREENSHOT OF PAGE 1 (CURRENT PAGE) ----------------
print(" Taking screenshot of Page 1...")
take_users_table_screenshot(1)
```

```
# ---------------- PAGINATION SCREENSHOTS (2, 3, 4) ----------------
for page in [2, 3, 4]:
    try:
        # Click pagination button
        page_btn = wait.until(

  EC.element_to_be_clickable((
                By.XPATH,
                f"//ul[contains(@class,'pagination')]//*[text()='{page}']"
            ))
        )

        driver.execute_script("arguments[0].click();", page_btn)
        print(f" Clicked page {page}")

        # Wait for new data to load
        time.sleep(4)

        # Take screenshot
        print(f" Taking screenshot of Page {page}...")
        take_users_table_screenshot(page)

    except TimeoutException:
        print(f"Page {page} not found or timed out")
    except Exception as e:
        print(f" Error on page {page}: {str(e)}")

# ---------------- SHOW SIDEBAR AGAIN ----------------
print("\n Making sidebar visible again...")
driver.execute_script("""
    const sidebar = document.querySelector('nav');
    if (sidebar) {
        sidebar.style.display = 'block';
    }
""")
time.sleep(1)
print("Sidebar is now visible")


# ----------------Click Logs----------------
logs_btn = wait.until(
    EC.element_to_be_clickable((
        By.XPATH,
        '//*[@id="kanso-nav-item-logsViewSection"]/div/div[2]'
    ))
)
driver.execute_script("arguments[0].click();", logs_btn)
print("Logs clicked")
time.sleep(1)
```

```
# ----------------Click Log Search----------------
log_search_btn = wait.until(
   EC.element_to_be_clickable((
      By.XPATH,
      '//*[@id="kanso-nav-item-logSearch"]/div/div/span'

 ))
)
driver.execute_script("arguments[0].click();", log_search_btn)
print("Log Search clicked")

#---------------- Click Query Input----------------
query_box = wait.until(
   EC.element_to_be_clickable((
      By.XPATH,
      '//*[@id="logs-query-input-container"]/div/div/div/div[2]/div[2]/div'
   ))
)
driver.execute_script("arguments[0].click();", query_box)
time.sleep(1)

query = """| json "EventID","SubjectDomainName","Message"as
EventID,SubjectDomainName,Message nodrop
| where EventID != null and Message != ""
| count as Count by EventID, SubjectDomainName, Message
| sort by Count
| limit 25"""

query_box.send_keys(Keys.CONTROL + "a")
query_box.send_keys(Keys.DELETE)
query_box.send_keys(query)
print("Query entered")

time.sleep(5)

# ----------------choosing the days----------------
timer = wait.until(
   EC.presence_of_element_located((By.XPATH, "//div[contains(text(), '-15m')]"))
)
driver.execute_script("arguments[0].click();", timer)
time.sleep(5)
s7days = wait.until(
   EC.presence_of_element_located((By.XPATH, "//div[@title='Last 7 Days']"))
)
driver.execute_script("arguments[0].click();", s7days)
time.sleep(5)
```

```
# ----------------searching logs----------------
search_btn = wait.until(
    EC.element_to_be_clickable((
        By.XPATH,
        '//*[@id="search-query-start-compact"]/div'
    ))
)

driver.execute_script("arguments[0].click();", search_btn)
print("Search started")

time.sleep(12)

#----------------Download CSV----------------
download_btn = wait.until(
    EC.element_to_be_clickable((
        By.XPATH,
        "//button[@title='Export CSV']"
    ))
)
driver.execute_script("arguments[0].click();", download_btn)
print("CSV Download started")

print(" LOGS DOWNLOADED SUCCESSFULLY")

chrome_options = Options()
chrome_options.add_argument("--start-maximized")
chrome_options.add_experimental_option("detach", True)


driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=chrome_options
)

driver.get("https://platform.securityscorecard.io/#/start")

wait = WebDriverWait(driver, 20)

time.sleep(5)
#email
email = driver.find_element(By.ID, "username")
email.send_keys("joyane6174@fftube.com")

#password
password = driver.find_element(By.ID, "password")
password.send_keys("Althafk@712")
```

```
#login button
log_in_btn = wait.until(
    EC.element_to_be_clickable((By.XPATH, '//*[@id="login-button"]'))
)

driver.execute_script("arguments[0].click();", log_in_btn)
print("signin clicked successfully")

#myorganisation
myorganisation_button = wait.until(
    EC.presence_of_element_located((By.XPATH,"//button[.//span[text()='My Organization']]"))
)
driver.execute_script("arguments[0].click();", myorganisation_button)
time.sleep(1)

#myscorecard
myscorecard_button = wait.until(
    EC.presence_of_element_located((By.XPATH,  "//a[.//span[text()='My Scorecard']]"))
)
driver.execute_script("arguments[0].click();", myscorecard_button)
time.sleep(10)

#fullscreenshot
driver.save_screenshot("myscorecard_fullpage.png")
print(" Full page screenshot saved")

#croppingprocess
full_path = "myscorecard_fullpage.png"
driver.save_screenshot(full_path)

#scorefactor
scorefactor_button = wait.until(
    EC.presence_of_element_located((By.XPATH,  "//a[contains(@href, '/factors')]"))
)

driver.execute_script("arguments[0].click();", scorefactor_button)
time.sleep(10)

driver.execute_script("document.body.style.zoom='50%'")
time.sleep(1)

#capturingtable
score_content = wait.until(
    EC.visibility_of_element_located((
        By.XPATH,
        "//main[@aria-label='Main workspace content area']//div[contains(@class, 'keWNRs')]"
    ))
)
time.sleep(10)
```

```
screenshot_path = "users_and_roles.png"
score_content.screenshot(screenshot_path)
print("Center content screenshot captured successfully")

def get_week_label(tool_name: str) -> str:
    today = datetime.date.today()
    week_num = (today.day - 1) // 7 + 1
    month_abbr = today.strftime("%b").lower()

    # Choose suffix
    if week_num == 1:
        suffix = "st"
    elif week_num == 2:
        suffix = "nd"

    elif week_num == 3:
        suffix = "rd"
    else:
        suffix = "th"

    return f"{week_num}{suffix}week_{month_abbr}_{tool_name.lower()}.png"

print("Image cropping")
# --- Overview crop ---
img1 = Image.open("myscorecard_fullpage.png")
width, height = img1.size
crop_box = (50, 120, 800, 360)
cropped = img1.crop(crop_box)

# Save with week label + tool name
filename1 = get_week_label("securityscorecard")
cropped.save(filename1)

# --- Score factors crop ---
img2 = Image.open("users_and_roles.png")
width1, height2 = img2.size
crop_box = (0, 0, 740, 522)
cropped = img2.crop(crop_box)

filename2 = get_week_label("score_factors")
cropped.save(filename2)

print("deleting old images")
# Clean up originals
os.remove("myscorecard_fullpage.png")
os.remove("users_and_roles.png")

driver.quit()
```

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

The project presents an effective solution to overcome the limitations of manual SOC reporting processes. In existing systems, data collection and report preparation are performed manually across multiple security platforms, which increases the time consumption and introduces the possibility of human errors.

The proposed system automates the complete workflow by integrating multiple modules such as authentication, data collection, screenshot capture, data processing, and report generation. Using Python and Selenium, the system is capable of accessing various platforms including Sumo Logic, Security Scorecard, Cisco Umbrella, and Trend Vision One, and extracting relevant security data in a structured manner.

The implementation of automation significantly reduces manual effort and ensures consistency in report generation. The integration of task scheduling further enhances the system by enabling periodic execution without user intervention. The generated reports provide a clear and organized representation of security insights, thereby supporting efficient analysis and decision-making.

Thus, the developed system improves the overall efficiency, accuracy, and reliability of SOC operations and serves as a practical solution for automated cybersecurity reporting.

6.2 FUTURE WORK

Although the proposed system achieves automation in SOC data collection and reporting, there is scope for further enhancement to improve its performance and scalability. One of the major improvements can be the integration of Application Programming Interfaces (APIs) for direct data retrieval, which would provide faster and more reliable execution compared to web automation.

The incorporation of Artificial Intelligence and Machine Learning techniques can enhance the system by enabling intelligent threat detection, anomaly identification, and predictive analysis. This would allow the system to move from a reactive approach to a proactive security solution.

In addition, the system can be extended to support real-time monitoring and dashboard visualization, providing live insights into security data. Deployment on cloud platforms can also be considered to improve scalability, accessibility, and centralized data management.

Further enhancements may include the development of a graphical user interface for ease of use and the implementation of advanced security mechanisms such as encrypted credential storage and access control. These improvements would make the system more robust and suitable for enterprise-level deployment.

CHAPTER 7

ANNEXURE

7.1 REFERENCES

Ajayi Toluwalope. *"Automating Security Operations Centers (SOCs) with AI: Benefits and Challenges"*, 2025.

Sarker, I.H., Abushark, Y.B., Alsolami, F., Khan, A.I. *"Machine Learning-Based Cybersecurity Intrusion Detection Model"*, IEEE, 2020.

Wagner, T.D., Mahbub, K., Palomar, E., Abdallah, A.E. *"Cyber Threat Intelligence Sharing: Survey and Research Directions"*, Computers & Security, 2019.

Karagiannis, S., Magkos, E. *"Comparative Analysis of Security Orchestration, Automation and Response (SOAR) Platforms"*, Information Journal, 2021.

Koch, R., Golling, M., Krieger, B. *"From Bro to Zeek: Network Security Monitoring"*, IEEE Symposium, 2019.

Docker Inc. *"Docker Documentation for Containerization and Deployment"*, 2024.

7.2 BIBLIOGRAPHY

https://www.researchgate.net

https://www.jetir.org

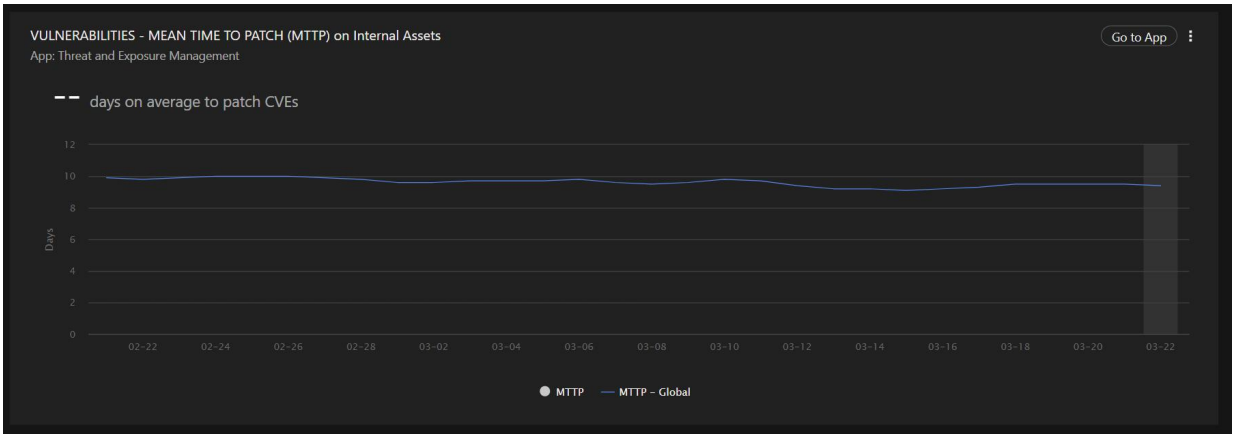
https://www.ieee.org

https://www.sciencedirect.com

https://arxiv.org

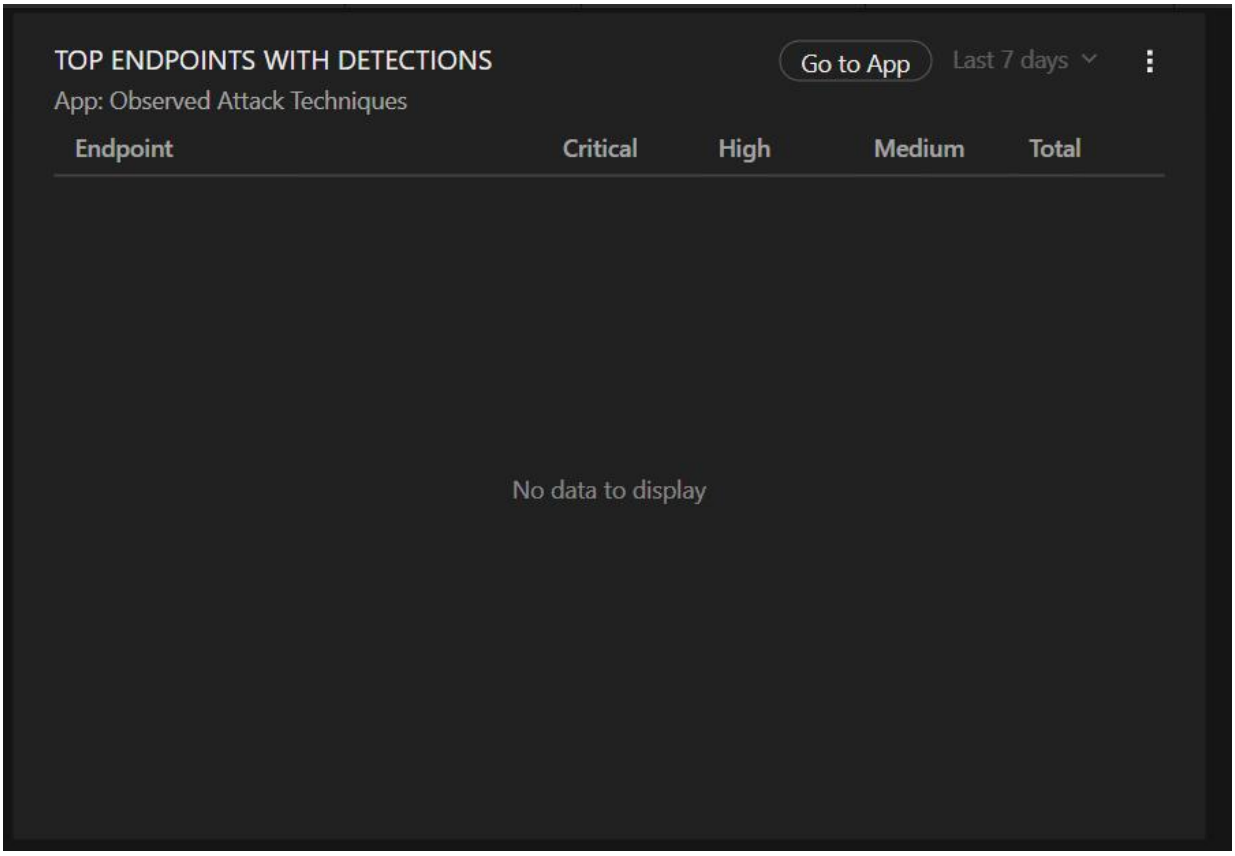
CHAPTER 8

APPENDIX

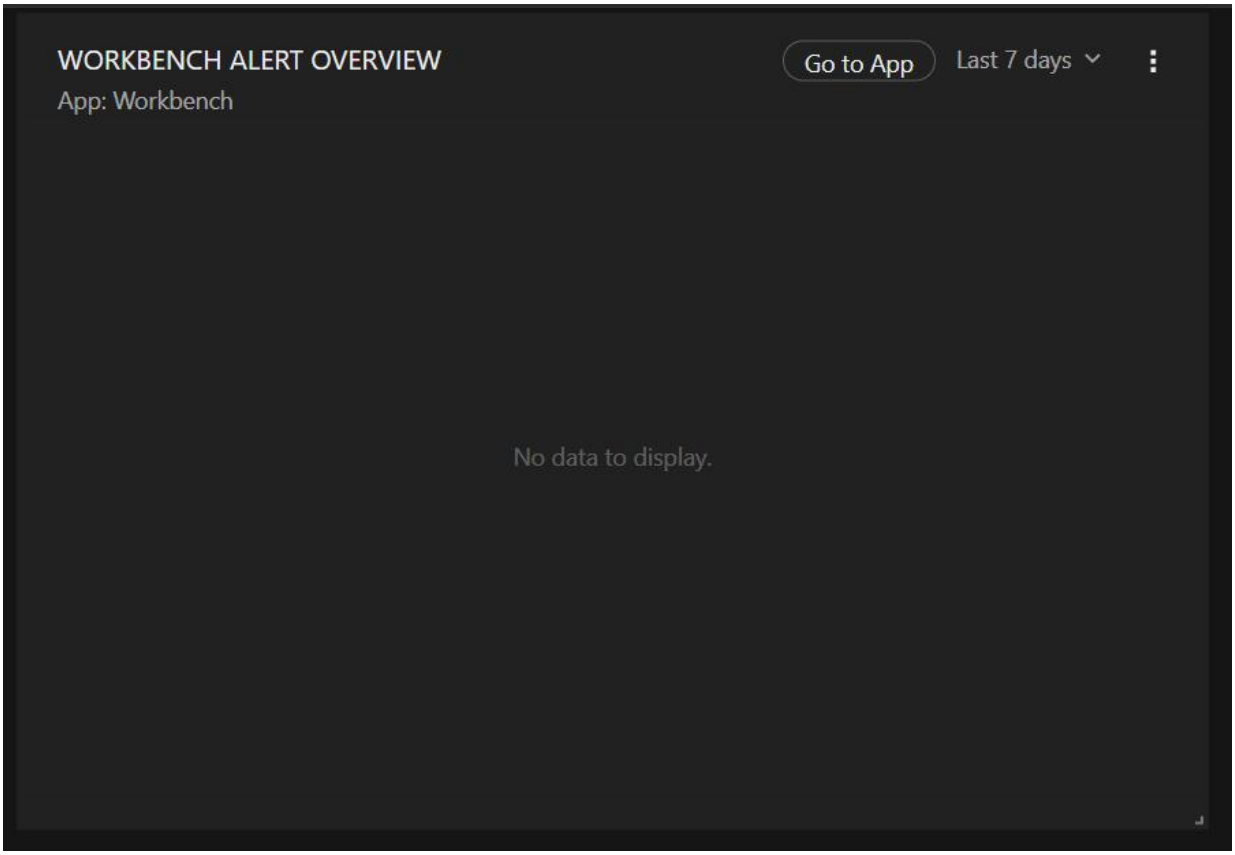


8.1 Mean time to patch

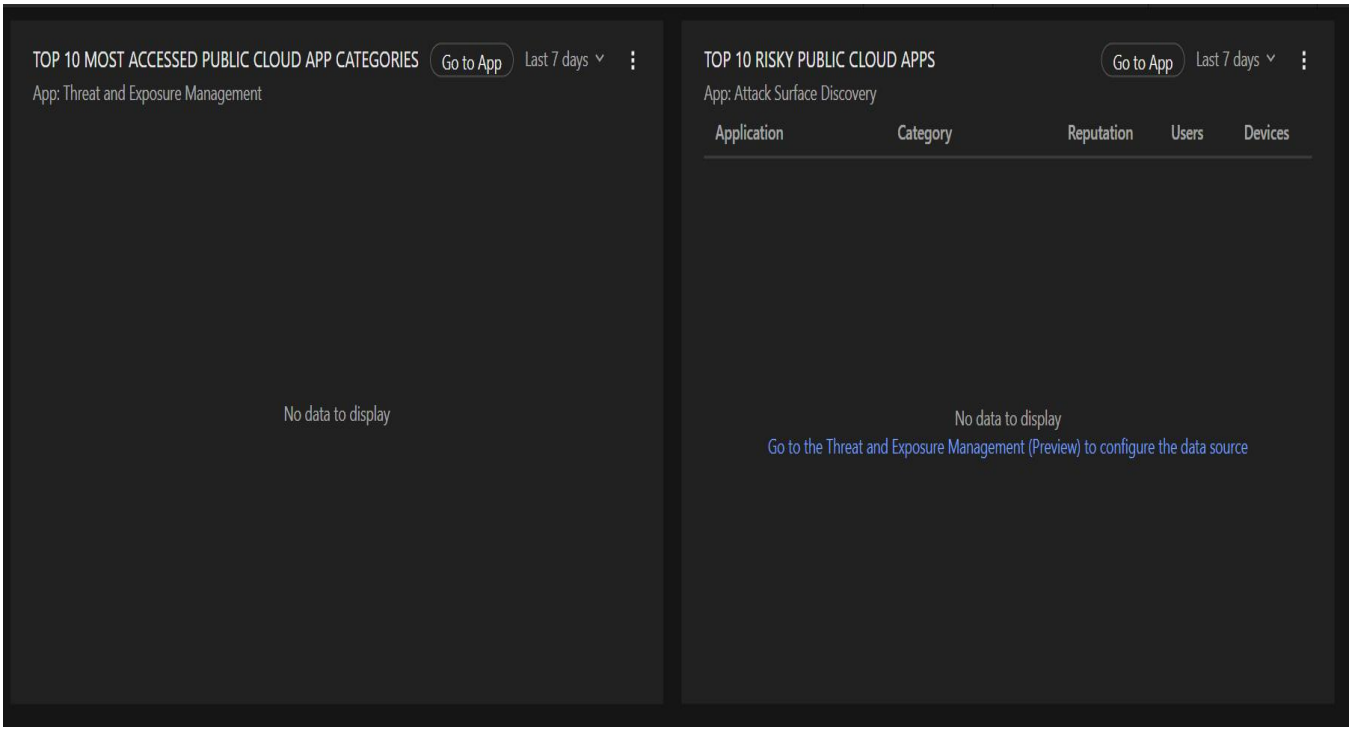
8.2 Average unpatched time



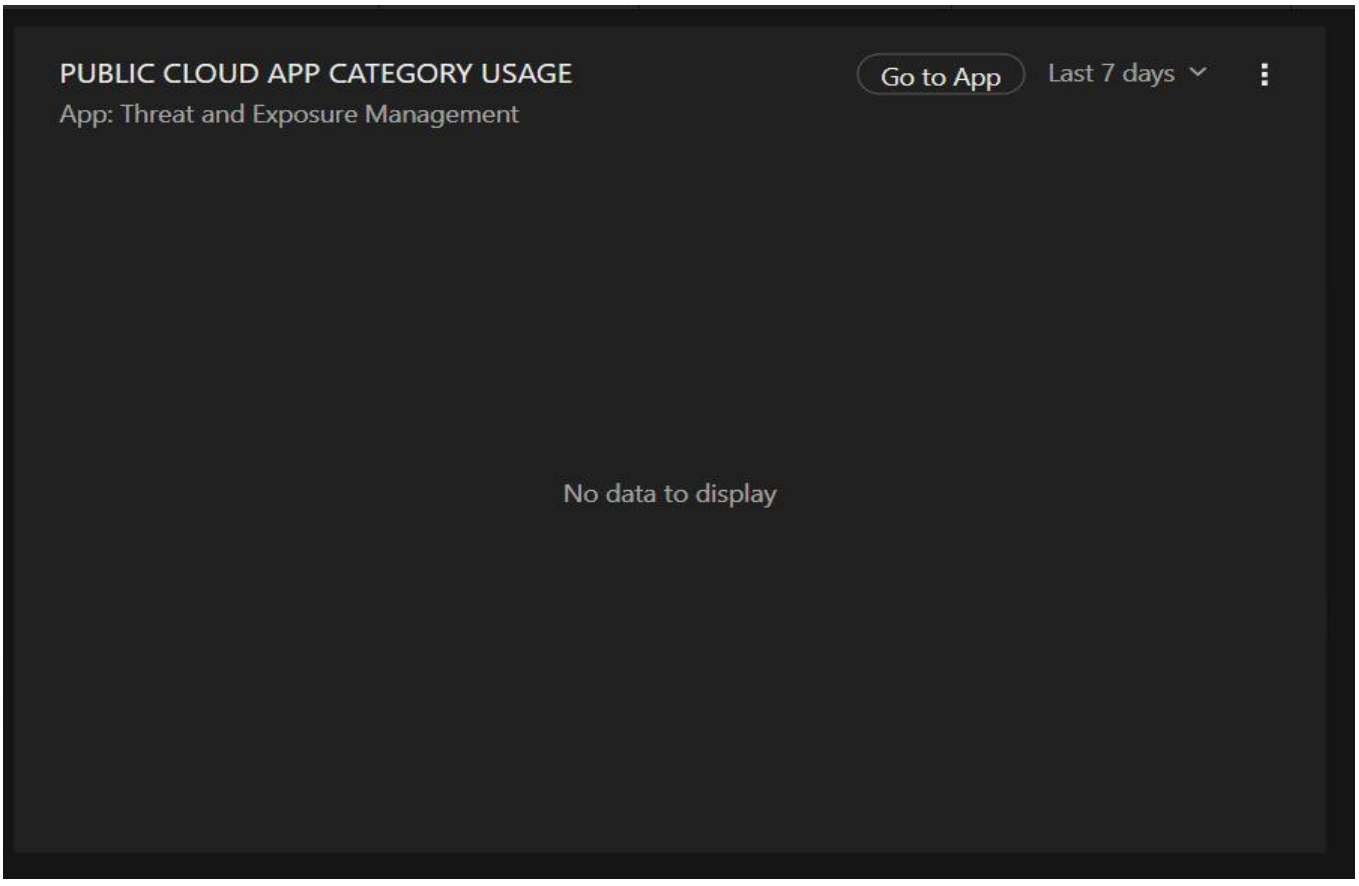
8.3 Top endpoints with detection



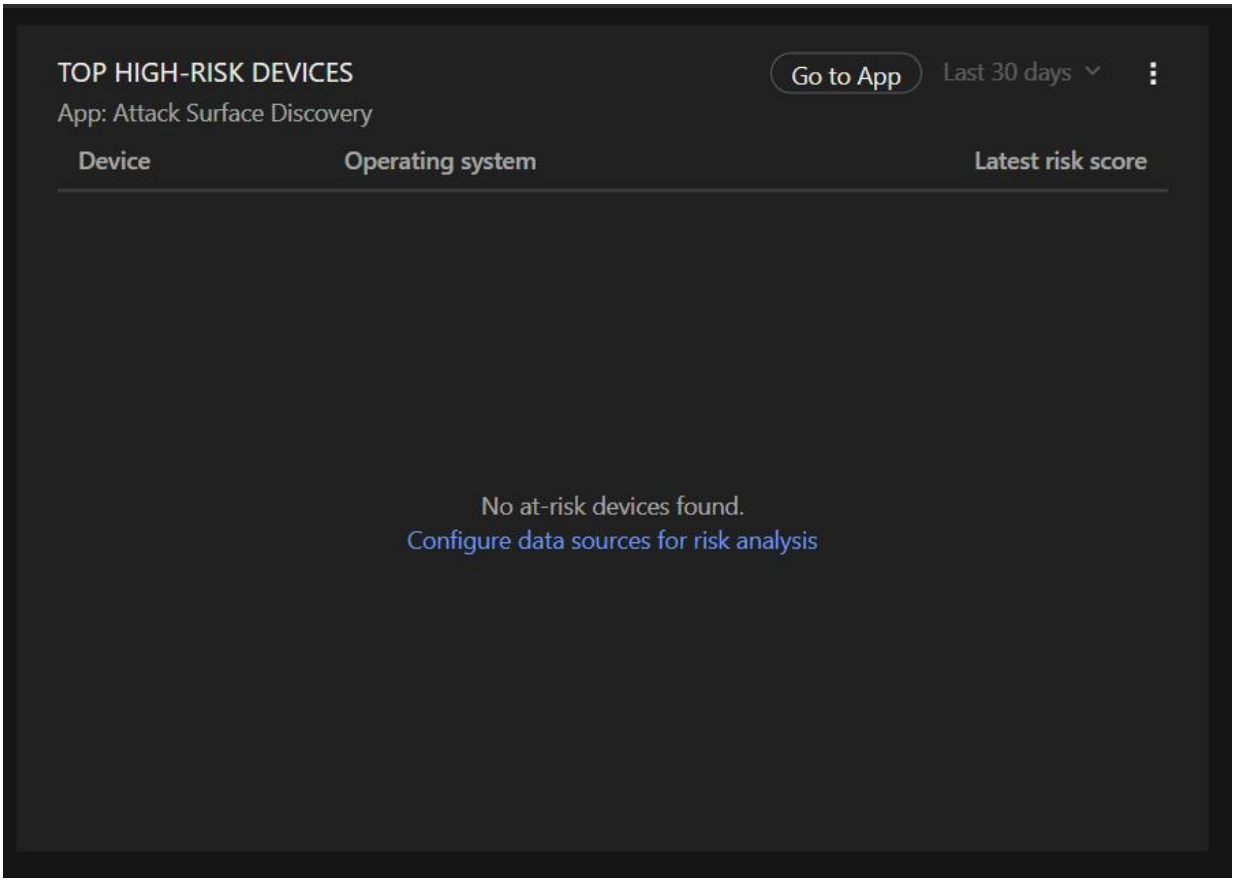
8.4 Workbench Alert



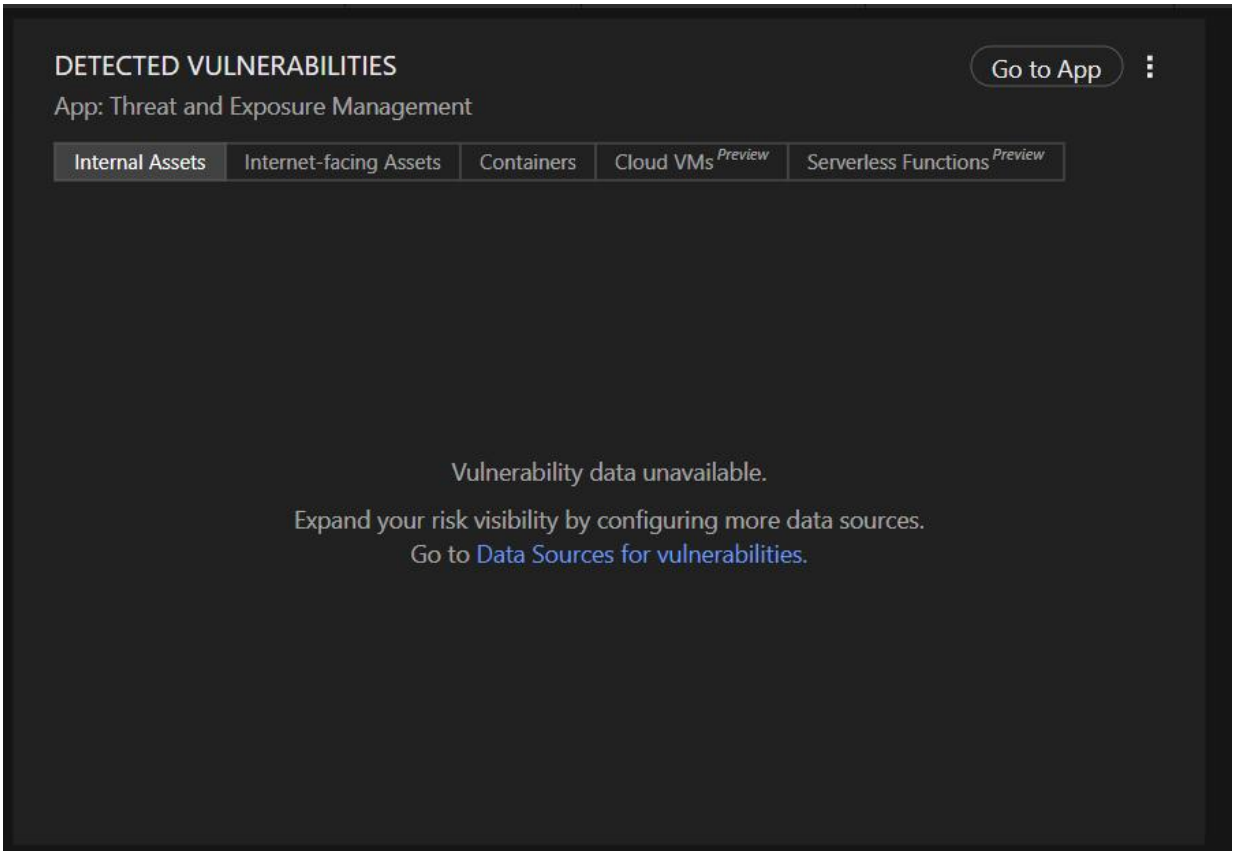
8.5 Top 10 most accessed cloud app categories and risky public cloud apps



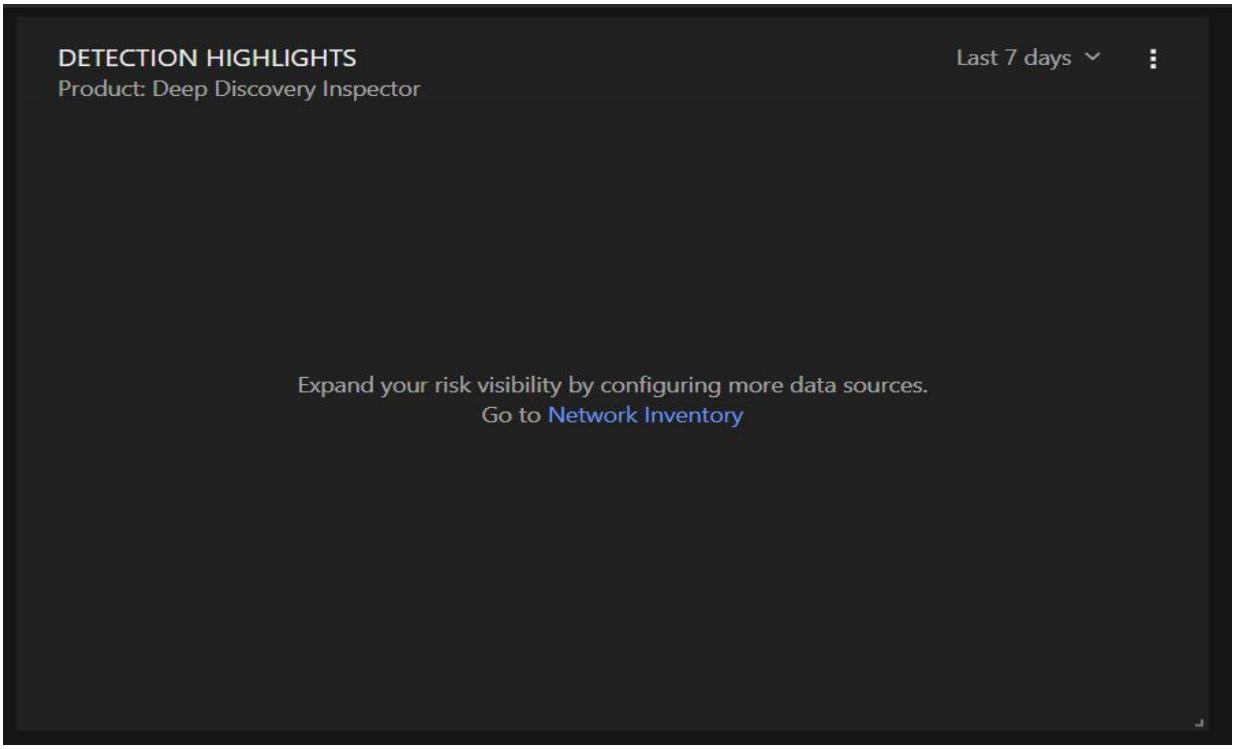
8.6 Public cloud app category usage



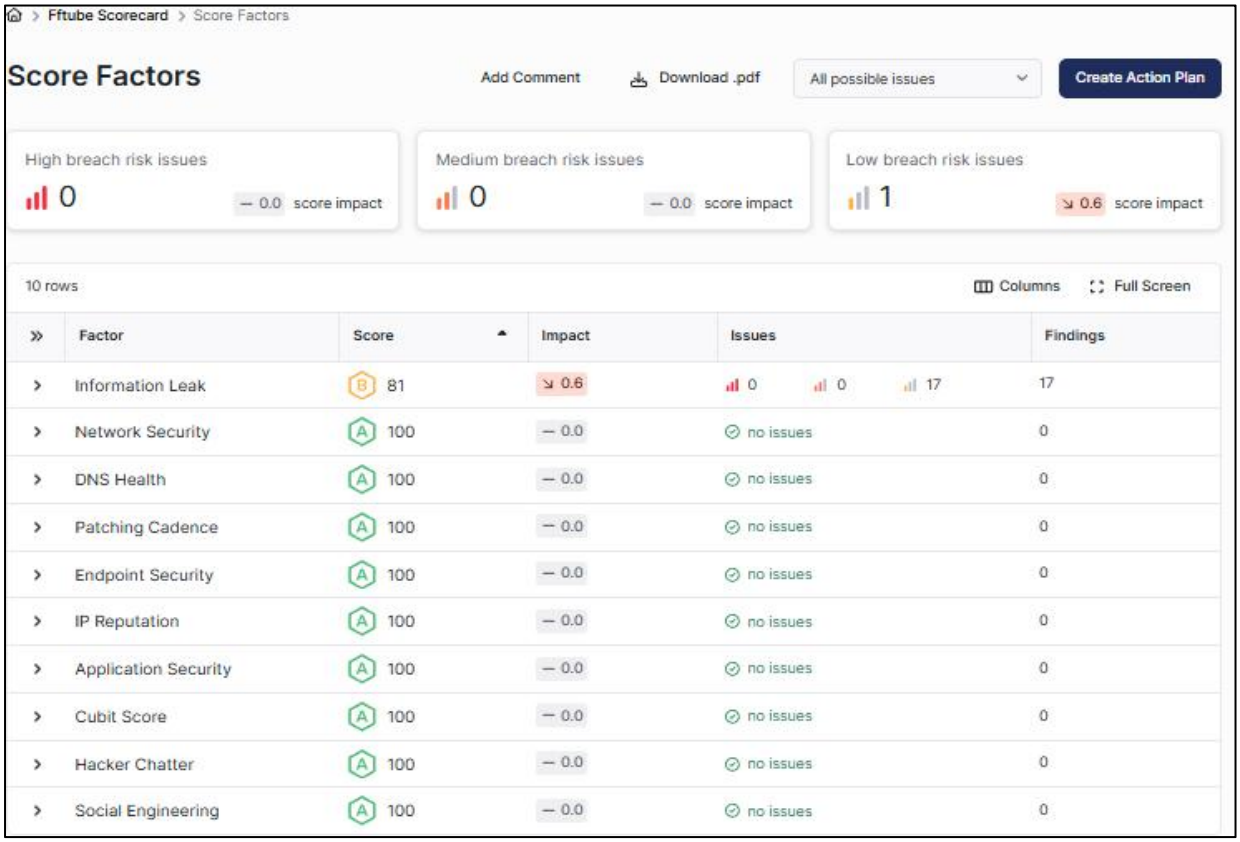
8.7 Top high risk devices



8.8 Detected vulnerabilities

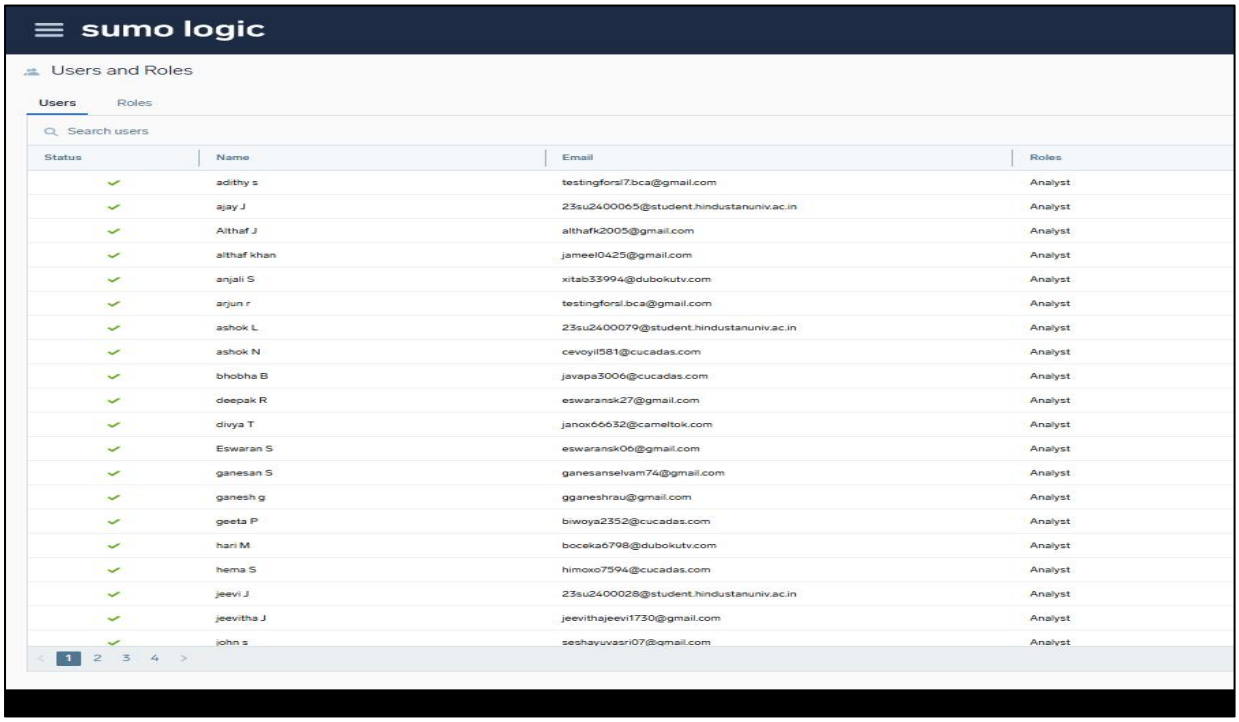


8.9 Detected highlights

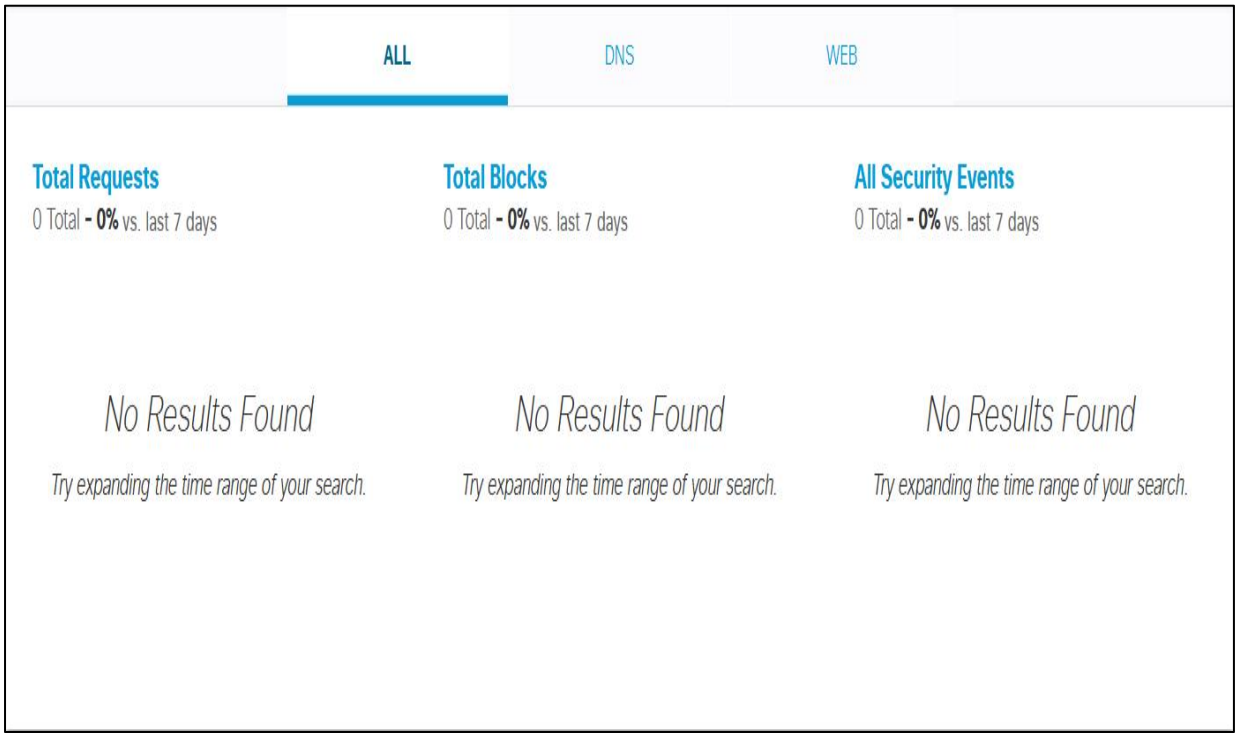


8.10 Score Factors

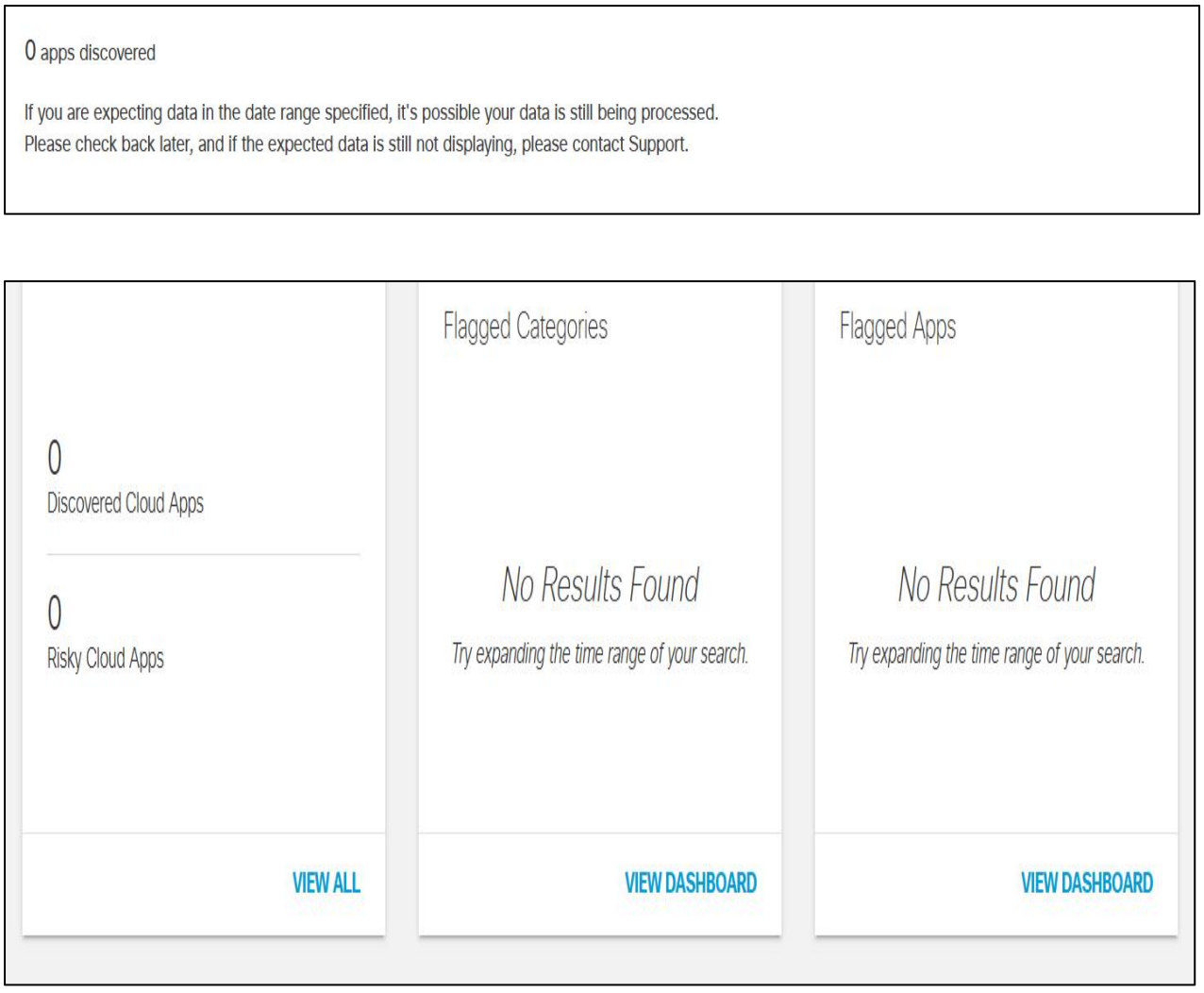
8.11 Organization Review



8.12 User and Roles



8.13 Network Breakdown



8.14 Flagged Apps

CHAPTER 9

AUTHOR & LICENSE

9.1 AUTHOR

Sesha S — Cybersecurity Graduate | SOC Analyst Aspirant | SIEM & Threat Detection Enthusiast

Sesha is a 2026 BCA Cybersecurity graduate with hands-on experience building SIEM-based monitoring solutions, automating SOC data collection and reporting workflows, and simulating real-world attack scenarios to validate defensive coverage. This project reflects practical, applied experience with security automation, log analysis, and report generation — core skills for an entry-level SOC Analyst role.

Core Skills: SIEM (Sumo Logic) · Python Security Automation · Selenium · Log Analysis · Windows Event Monitoring · Burp Suite · Nmap · Metasploit

Connect: LinkedIn: linkedin.com/in/sesha-s-218513368 | GitHub: github.com/seshaselvam07-spec

9.2 LICENSE